From measuring imbalance to fixing it

You can now **score** a classifier on imbalanced data (L12: PR-AUC, recall, MCC) and **trust** its probabilities (L13: calibration). But the cheese detector still *misses most of the 3% bad batches*.

Can we do better by changing the **data** or the **training** itself — not just the metric we report? This deck: the honest answer.

Two families of fixes: **tilt the training** (class weights / costs) and **reshape the data** (over/under-sampling, SMOTE). Both help *sometimes* — and both have traps.

Predict-first: just rebalance the classes?

Take the cheese data (3% bad) and **duplicate the bad batches until the classes are 50/50**, then refit. Predict three things:

(a) recall at the default threshold 0.5? **(b)** the ranking (ROC/PR-AUC)? **(c)** the predicted probabilities?

- **(a) Recall jumps** — the decision boundary shifts toward the minority.
- **(b) Ranking barely moves** (if anything, a touch worse) — copies add no new information.
- **(c) Probabilities inflate** — the model now thinks "bad" is common; they **decalibrate**. (And if you duplicated *before* the train/test split, you leaked.)

The recall gain in (a) is real — but you could have gotten it for free by **lowering the threshold** (L12). That tension is the whole lecture.

Outline

What you already have

Tilt the training: weights & costs

Reshape the data: resampling

Does it actually help?

Start with what you already have

Before reshaping data, remember: two tools from L12–L13 handle most imbalance already.

Why imbalance hurts — and your two existing tools

Why it hurts: training minimizes *average* loss, which is dominated by the majority — so the model can score well by mostly ignoring the rare class (L12: 97% accuracy, 0 recall).

You already have two fixes (L12–L13):

- the **right metric** — PR-AUC, recall, MCC, balanced accuracy;
- the **cost-tuned threshold** — move c off 0.5 toward the minority.

Reach for data tricks only after these. Often the metric + threshold are enough — they cost nothing and keep your probabilities honest.

A map of the approaches

Family	**Idea**	**Where**
Algorithm-level	reweight errors / cost matrix in training	**this deck**
Data-level	resample to rebalance the classes	**this deck** (in depth)
Cost-sensitive *thresholding*	move the cutoff by cost	**done in L12/L13**
Ensemble	imbalance-aware bagging/boosting	**ch. 4 (trees)**

This deck covers the first two; the threshold is already yours; imbalance-specific ensembles (BalancedRF, RUSBoost, EasyEnsemble) come with trees in chapter 4.

Tilt the training: weights & costs

Tell the learner that minority mistakes hurt more — without touching the data.

Class weights in the loss

Weight each example's loss by its class, so minority errors count more:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} w_{y^{(i)}} \, L\big(y^{(i)}, f(\mathbf{x}^{(i)})\big), \qquad w_k \propto \frac{n}{K \, n_k}.$$

- `class_weight="balanced"` sets exactly $w_k = n/(K\, n_k)$ — the rare class is up-weighted automatically.
- It is just **weighted ERM** — works for any loss-based learner (callback Ch. 1).

Same machinery as logistic regression (L11), only the per-example weights change.

The catch: it's roughly a threshold move — and it decalibrates

- For a *well-specified* model, reweighting is **approximately** a shift of the prior / decision threshold (Elkan, 2001) — not a brand-new model.
- With **regularization** the fitted coefficients genuinely change, so it is not an *exact* threshold move — but close.
- Crucially, it **decalibrates**: the model trains as if the minority were common, so its probabilities **inflate** (callback L13).

In the bank-marketing project, `class_weight="balanced"` blew ECE from 0.01 to 0.30. For a probabilistic model it mostly *duplicates threshold tuning at the cost of calibration* — if you use it and report probabilities, **recalibrate**.

It *does* change the fit meaningfully for tree split / SVM margin criteria — more on "when it helps" later.

Cost-sensitive learning — just the idea

L12/L13 put costs at the **decision threshold**. You can instead bake a **cost matrix** into **training**, so the model itself minimizes expected cost:

		true	
		bad (y=1)	good (y=0)
pred.	bad	0	C_{FP}
	good	C_{FN}	0

- For a rare, costly positive: $C_{FN} \gg C_{FP}$ (a missed bad batch $\gg$ a re-check).
- **Class weights are the simple special case** — one cost per class.

Full methods (MetaCost, instance-specific costs) exist but are beyond this course — the *idea* is what to remember.

Reshape the data: resampling

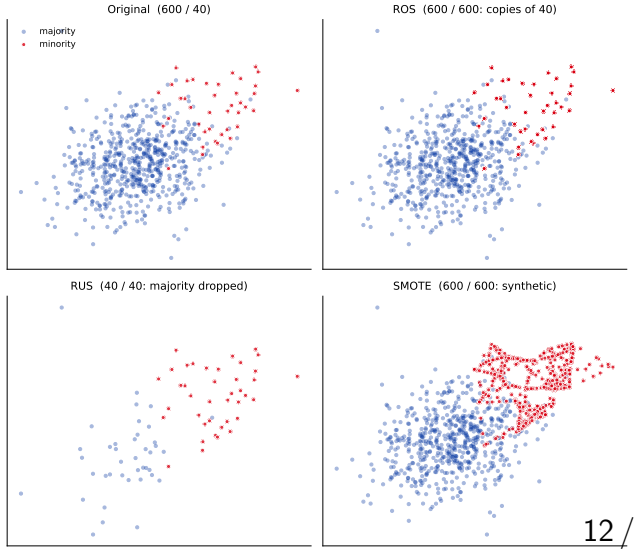
Rebalance the training set itself — classifier-independent, flexible, and the most widely used family. We go deep here; keep the "threshold first" ordering in mind.

Under-, over-, and hybrid sampling

Three ways to rebalance **the training data**:

- **Undersampling** — drop majority examples.
- **Oversampling** — add/synthesize minority examples.
- **Hybrid** — both.

Independent of the classifier ⇒ flexible. See what each does to one blob:

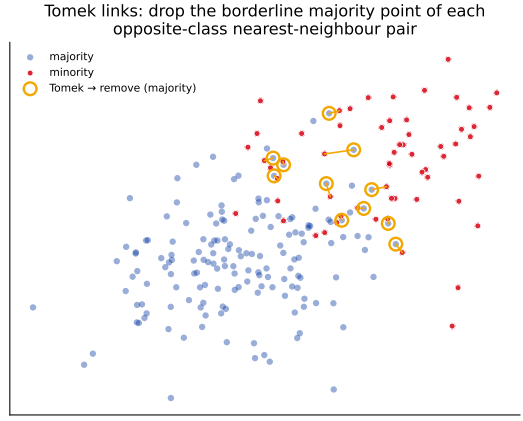


Undersampling: random, then smarter

Random undersampling (RUS) — drop majority points at random. Fast, but may **throw away informative examples**.

Tomek links (smarter) — a pair of *opposite-class nearest neighbours*. They sit on the boundary, so drop the **majority** one to clean it up.

Other cleaning rules exist (edited-NN / NCL, CNN, one-sided selection) — same spirit: remove majority points near the border.



Oversampling: random replication (ROS)

Random oversampling (ROS) — duplicate minority examples until balanced.

- Cheap, classifier-independent, loses no data.
- **But: exact copies** $\Rightarrow$ the model can **overfit** those few points (it sees the same example many times, not new evidence).

Can we add minority examples that are *plausible but new*, instead of copies? That is the idea behind **SMOTE**.

SMOTE: synthesize, don't copy

SMOTE (Synthetic Minority Over-sampling) makes *new* minority points by **interpolating between neighbours**: for a minority point $\mathbf{x}^{(i)}$ and one of its minority neighbours $\mathbf{x}^{(j)}$,

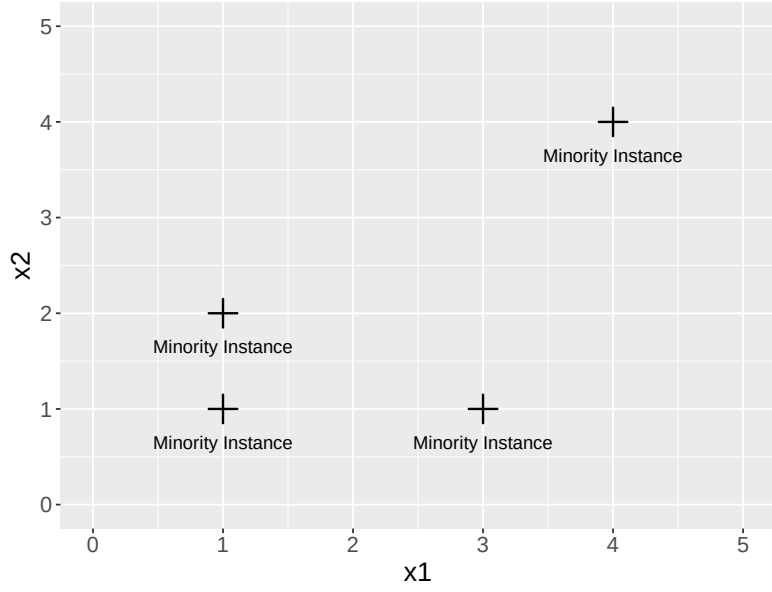
$$\mathbf{x}_{\text{new}} = \mathbf{x}^{(i)} + \lambda \left(\mathbf{x}^{(j)} - \mathbf{x}^{(i)}\right), \qquad \lambda \in [0, 1].$$

By hand: take $\mathbf{x}^{(i)} = (1, 2)$, neighbour $\mathbf{x}^{(j)} = (3, 1)$, and $\lambda = 0.25$:

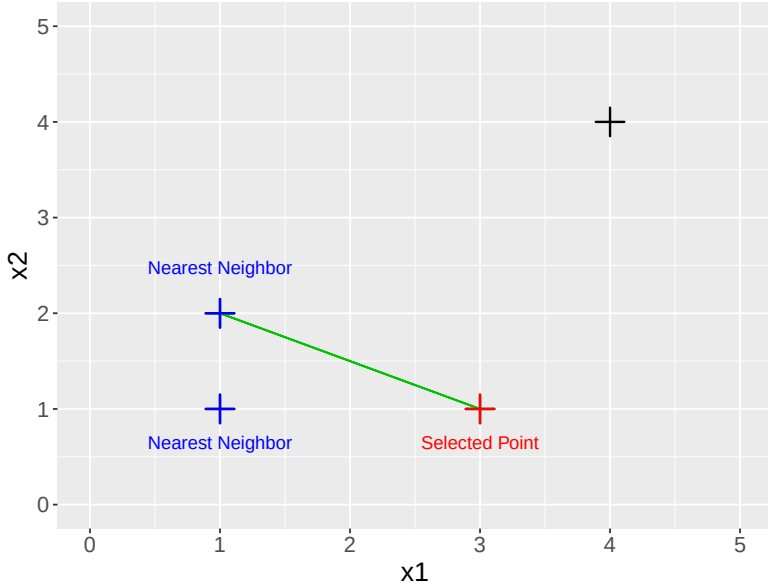
$$\mathbf{x}_{\text{new}} = (1, 2) + 0.25\,(3 - 1,\ 1 - 2) = (1, 2) + 0.25\,(2, -1) = (\mathbf{1.5},\ \mathbf{1.75}).$$

A point a quarter of the way from $\mathbf{x}^{(i)}$ toward its neighbour — plausible, and *new* (not a copy). The next slide builds these step by step.

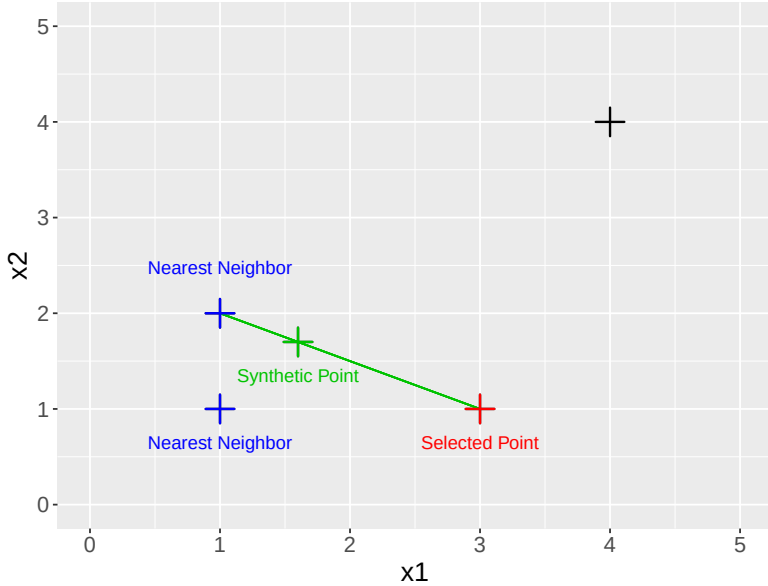
SMOTE: building the points (step through)



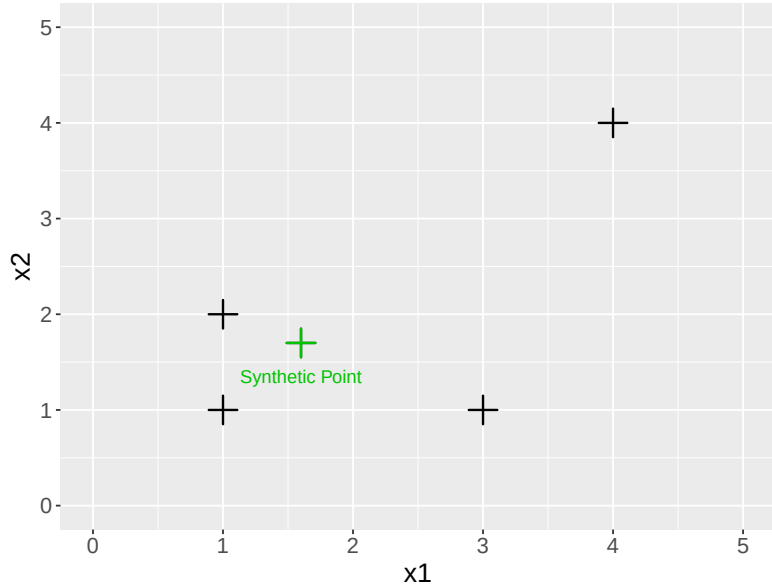
SMOTE: building the points (step through)



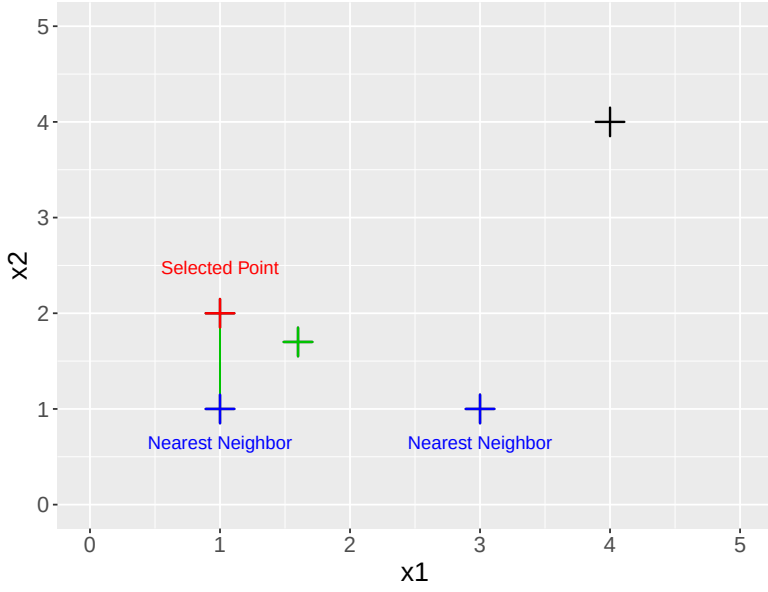
SMOTE: building the points (step through)



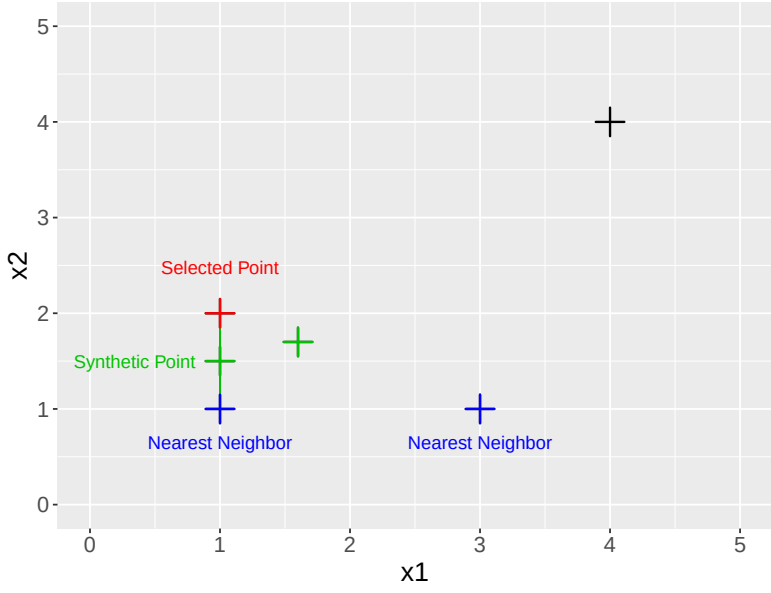
SMOTE: building the points (step through)



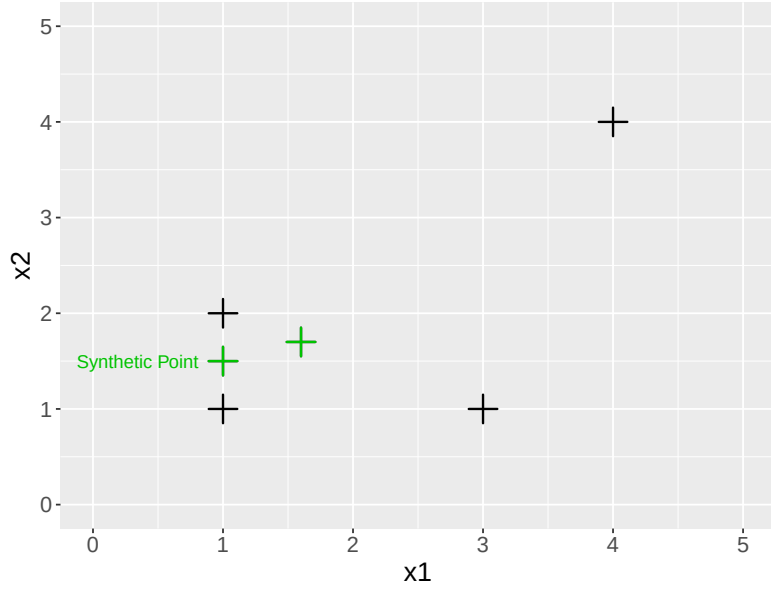
SMOTE: building the points (step through)



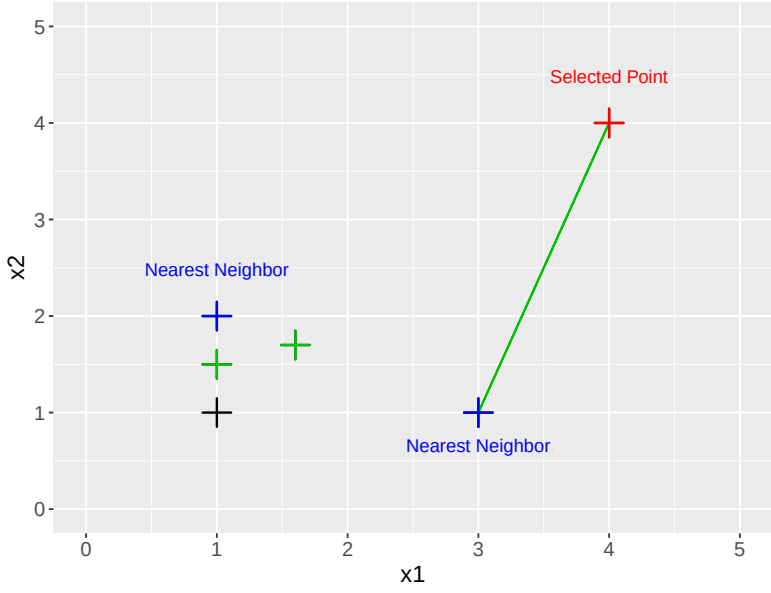
SMOTE: building the points (step through)



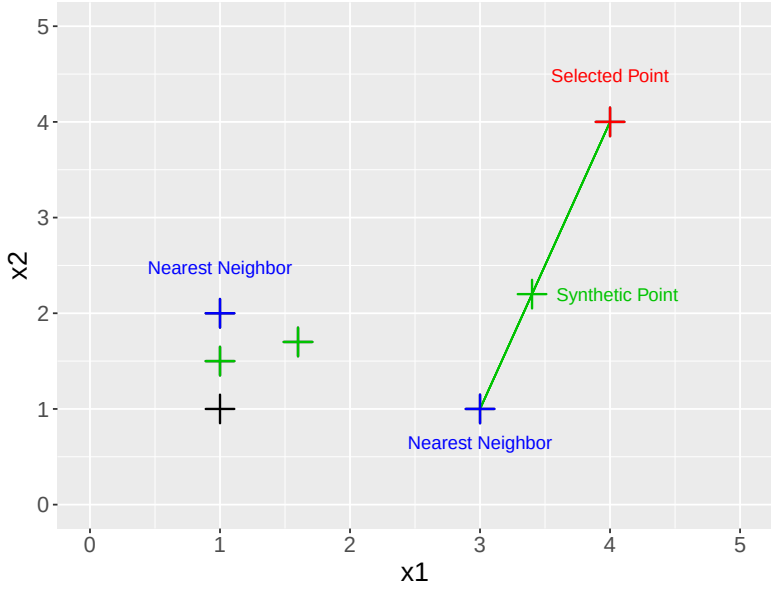
SMOTE: building the points (step through)



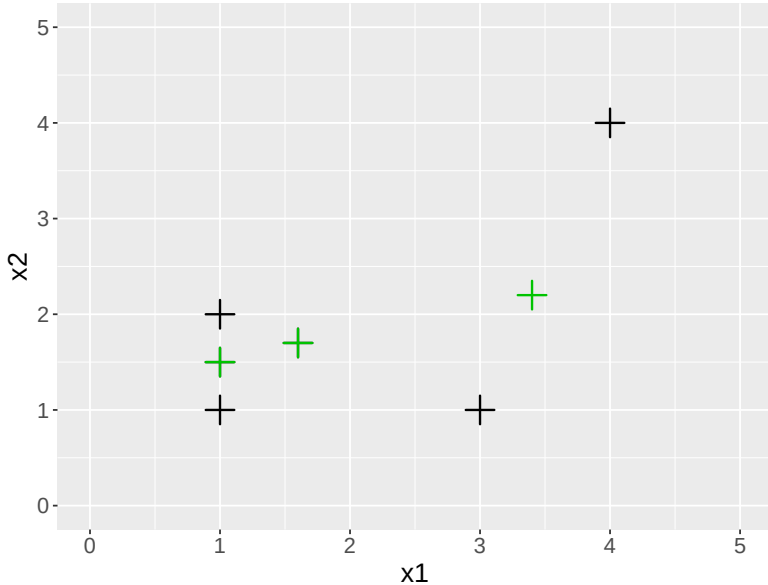
SMOTE: building the points (step through)



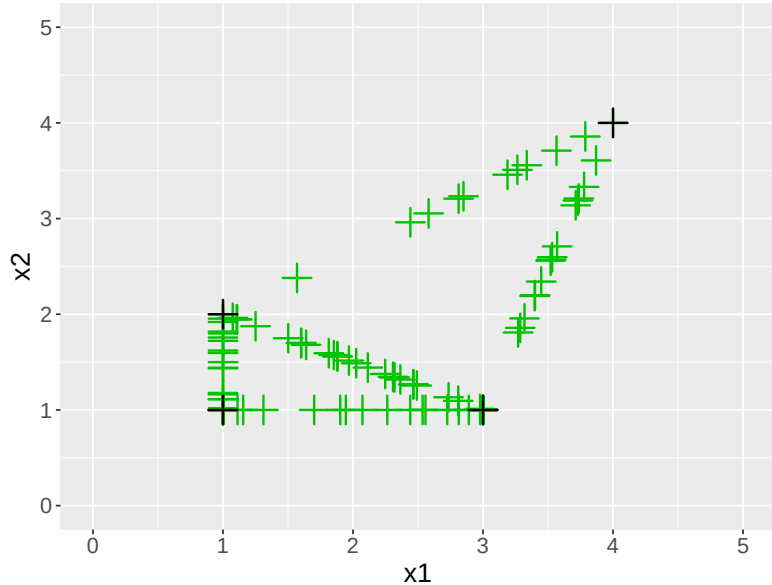
SMOTE: building the points (step through)



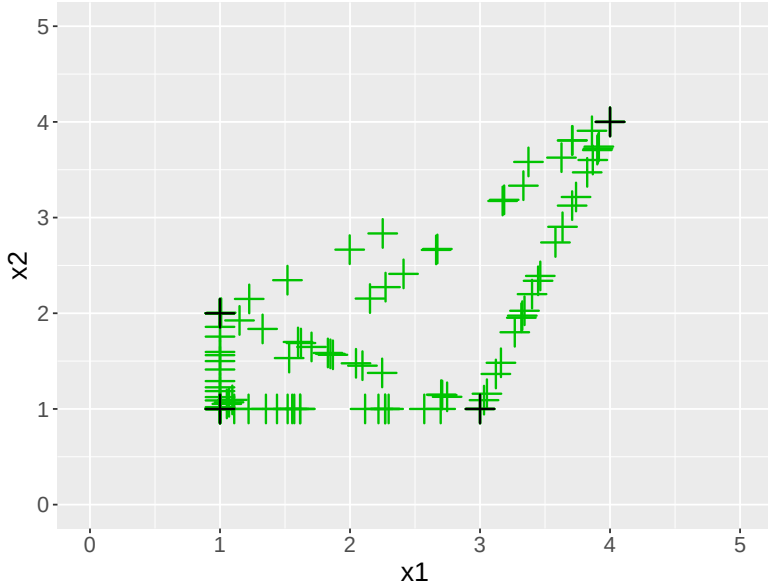
SMOTE: building the points (step through)



SMOTE: building the points (step through)



SMOTE: building the points (step through)



SMOTE in practice

Pros

- Generalizes the minority *region* instead of memorizing a few points (vs ROS).
- The basis for 90+ variants (Borderline-SMOTE, ADASYN, . . .).

Cons

- Ignores the majority ⇒ synthetic points can **bleed across the boundary** into majority territory.
- **Numeric-only** (it interpolates).

Hybrid: pair SMOTE with a cleaning step — **SMOTETomek** (SMOTE then Tomek) or **SMOTEENN** (SMOTE then edited-NN): oversample, then tidy the border.

Categoricals? Plain SMOTE can't interpolate "job = student". Use **SMOTENC** for mixed numeric/categorical data — e.g. the bank-marketing project, with its 9 categorical columns.

Resampling in code — on the training fold only

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline          # NOTE: imblearn's
    Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("smote", SMOTE(random_state=509)),          # resamples TRAIN
        folds only
    ("clf",   LogisticRegression(max_iter=2000)),
])
cross_val_score(pipe, X, y, scoring="average_precision", cv=5)
# class weights instead of resampling:
LogisticRegression(class_weight="balanced")
```

Use `imblearn`'s `Pipeline` so the sampler runs **inside each CV fold** — never resample before splitting (next section: why).

Does it actually help?

When resampling genuinely beats a tuned threshold — and the two traps that bite if you're careless.

When it genuinely beats thresholding

Reweighting / resampling helps when imbalance corrupts the **fit itself**, not just the operating point:

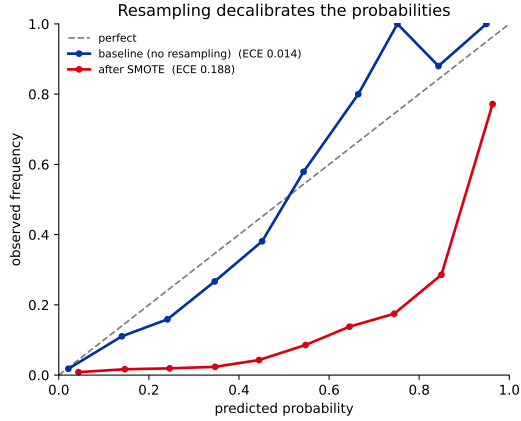
- the minority region is **too sparse for the model to carve a boundary** there;
- **regularization** shrinks the weak minority signal away;
- a **tree's split criterion** picks different splits once the minority is up-weighted.

Rule of thumb: if moving the threshold fixes it, you didn't need the data trick. If the model never *learned* the minority region, resampling/weights might actually help.

Two traps: leakage and decalibration

1. Leakage. Resample **only the training fold**, never before the split — otherwise synthetic/duplicated points leak across train/test and the score is fantasy. Use `imblearn.Pipeline` (previous slide).

2. Decalibration. Resampling changes the base rate the model sees $\Rightarrow$ its probabilities inflate. **Recalibrate** on untouched data (L13), or report rankings only.

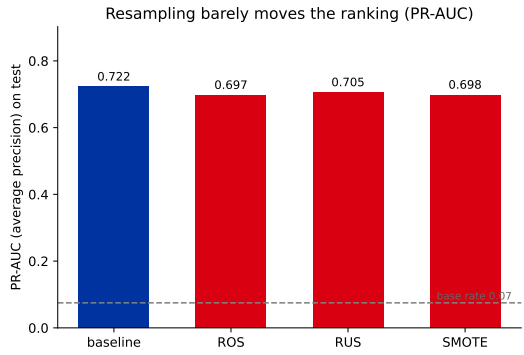


Reality check — on our own data

Same imbalanced data, same logistic regression, four pre-processings. Compare **PR-AUC** (ranking quality, threshold-free):

- Resampling **barely moves** it — here it even dips slightly.
- It changes *recall at a fixed threshold*, but that's a threshold move in disguise.

The effect is **small and not even reliably positive** — a tiny gain sometimes (literature: Optdigits F1 0.92 → 0.95), nothing or slightly worse other times (here). Trees/boosting (ch. 4) handle imbalance more natively.



Decision guide

In order:

1. **Right metric + cost-tuned threshold** (L12/L13) — almost always start here.
2. `class_weight` if your learner uses it — then **recalibrate**.
3. **Resampling / SMOTE** only if the minority is truly tiny **and the fit (not the threshold) is the bottleneck** — *inside* CV, then recalibrate.
4. **Get more minority data** / reframe the problem.
5. **Tabular?** reach for trees & boosting (ch. 4), which handle imbalance well.

Resampling is a tool, not a cure — powerful when the fit is starved, wasteful when the threshold would have done the job.

Recap

- Imbalance hurts because average loss is dominated by the majority — but you already fix most of it with the **right metric** + a **cost-tuned threshold** (L12/L13).
- **Class weights / cost-sensitive learning** tilt the training; for a probabilistic model they mostly mimic a threshold move and **decalibrate** $\Rightarrow$ recalibrate.
- **Resampling**: RUS / Tomek (undersample), ROS (copies $\rightarrow$ overfit), **SMOTE** (synthetic interpolation; SMOTENC for categoricals).
- Two traps: **resample inside the fold** (leakage) and **recalibrate** (decalibration). Real gains are usually **small**.

Next: decision trees & ensembles (ch. 4) — models that handle non-linearities and imbalance more natively, and bring imbalance-aware ensembles (BalancedRF, RUSBoost).